Fonctions

Lorsqu'une tâche doit être réalisée plusieurs fois par un programme avec seulement des paramètres différents, on peut l'isoler au sein d'une fonction.

Cette approche est également intéressante si la personne qui définit la fonction est différente de celle qui l'utilise. Par exemple, nous avons déjà utilisé la fonction **print()** qui avait été définie par quelqu'un d'autre.

Nous avons déjà rencontré diverses fonctions prédéfinies : **print**(), **input**(), **range**(), **len**().

I) DEFINITION D'UNE FONCTION :

La syntaxe Python pour la définition d'une fonction est la suivante :

```
def nom_fonction(liste de paramètres):
   bloc d'instructions
```

Vous pouvez choisir n'importe quel nom pour la fonction que vous créez, à l'exception des mots-clés réservés du langage, et à la condition de n'utiliser aucun caractère spécial ou accentué (le caractère souligné « _ » est permis). Comme c'est le cas pour les noms de variables, on utilise par convention des minuscules, notamment au début du nom (les noms commençant par une majuscule seront réservés aux classes).

1°) Corps de la fonction :

Comme les instructions **if**, **for** et **while**, l'instruction **def** est une instruction composée. La ligne contenant cette instruction se termine obligatoirement par un deux-points :, qui introduisent un bloc d'instructions qui est précisé grâce à l'indentation. Ce bloc d'instructions constitue le **corps de la fonction**.

2°) Fonction sans paramètre :

```
def compteur3():
   i = 0
   while i < 3:
      print(i)
      i = i + 1
print("bonjour")
compteur3()
compteur3()
```

Affichage après exécution :

```
>>>
bonjour
0
1
2
0
1
2
```

En entrant ces quelques lignes, nous avons défini une fonction très simple qui compte jusqu'à 2. Notez bien les parenthèses, les deux points, et l'indentation du bloc d'instructions qui suit la ligne d'en-tête (c'est ce bloc d'instructions qui constitue le corps de la fonction proprement dite).
Après la définition de la fonction, on trouve le programme principal qui débute par l'instruction **print("bonjour")**. Il y a ensuite au sein du programme principal, l'appel de la fonction grâce à **compteur3().**
Nous pouvons maintenant réutiliser cette fonction à plusieurs reprises, autant de fois que nous le souhaitons.

Nous pouvons également l'incorporer dans la définition d'une autre fonction.

```
def compteur3():
    i = 0
    while i < 3:
        print(i)
        i = i + 1
def double_compteur3():
    compteur3()
    compteur3()
print("bonjour")
double_compteur3()
```

Une première fonction peut donc appeler une deuxième fonction, qui elle-même en appelle une troisième, etc.
Créer une nouvelle fonction offre l'opportunité de donner un nom à tout un ensemble d'instructions. De cette manière, on peut simplifier le corps principal d'un programme, en dissimulant un algorithme secondaire complexe sous une commande unique, à laquelle on peut donner un nom explicite.
Une fonction est donc en quelque sorte une nouvelle instruction personnalisée, qu'il est possible d'ajouter librement à notre langage de programmation.

3°) Fonction avec paramètre :

```
def compteur(stop):
    i = 0
    while i < stop:
        print(i)
        i = i + 1
compteur(4)
compteur(2)
```

Affichage après exécution :

```
>>>
0
1
2
3
0
1
```

4°) Utilisation d'une variable comme argument :

L'argument que nous utilisons dans l'appel d'une fonction peut être une variable.

```
def compteur(stop):
    i = 0
    while i < stop:
        print(i)
        i = i + 1
a = 5
compteur(a)
```

Affichage après exécution :

```
>>>
0
1
2
3
4
```

Dans l'exemple ci-dessus, l'argument que nous passons à la fonction compteur() est le contenu de la variable a. A l'intérieur de la fonction, cet argument est affecté au paramètre stop, qui est une tout autre variable.
Notez donc bien dès à présent que :

- Le nom d'une variable que nous passons comme argument n'a rien à voir avec le nom du paramètre correspondant dans la fonction.
- Ces noms peuvent être identiques si vous le voulez, mais vous devez bien comprendre qu'ils ne désignent pas la même chose (en dépit du fait qu'ils puissent contenir une valeur identique).

5°) Fonction avec plusieurs paramètres :

La fonction suivante utilise trois paramètres : **start** qui contient la valeur de départ, **stop** la borne supérieure exclue comme dans l'exemple précédent et **step** le pas du compteur.

```
def compteur_complet(start, stop, step):
   i = start
   while i < stop:
      print(i)
      i = i + step
compteur_complet(1, 7, 2)
```

Affichage après exécution :

```
>>>
1
3
5
```

Pour définir une fonction avec plusieurs paramètres, il suffit d'inclure ceux-ci entre les parenthèses qui suivent le nom de la fonction, en les séparant à l'aide de virgules.
Lors de l'appel de la fonction, les arguments utilisés doivent être fournis dans le même ordre que celui des paramètres correspondants (en les séparant eux aussi à l'aide de virgules). Le premier argument sera affecté au premier paramètre, le second argument sera affecté au second paramètre, et ainsi de suite.

6°) Fonction lambda :

Il existe également la possibilité de créer des fonctions courtes en utilisant l'instruction lambda.
Exemple :

```
>>> f = lambda x, y: x + y
>>> f(6,8)
14
```

Python permet une syntaxe intéressante qui vous laisse définir des mini-fonctions d'une ligne à la volée. Ces fonctions dites lambda peuvent être employées partout où une fonction est nécessaire.
Présentation des fonctions lambda :

```
>>> def f(x):
...     return x*2
...
>>> f(3)
6
>>> g = lambda x: x*2 ❶
>>> g(3)
6
>>> (lambda x: x*2)(3) ❷
6
```

❶ Voici une fonction lambda qui fait la même chose que la fonction ordinaire précédente. On peut noter la syntaxe condensée : il n'y a pas de parenthèses autour de la liste d'arguments et le mot-clé return est manquant (il est implicite, la fonction complète ne pouvant être qu'une seule expression). La fonction n'a pas de nom, mais qu'elle peut être appelée à travers la variable à laquelle elle est assignée.

❷ On peut utiliser une fonction lambda sans l'assigner à une variable. Ce n'est pas forcément très utile, mais cela démontre qu'une fonction lambda est simplement une fonction en ligne.

Plus généralement, une fonction lambda est une fonction qui prend un nombre quelconque d'arguments et retourne la valeur d'une expression unique. Les fonctions lambda ne peuvent pas contenir de commandes et elles ne peuvent contenir plus d'une expression. Ne pas essayer de mettre trop de choses dans une fonction lambda. En cas de besoin de quelque chose de complexe, définir plutôt une fonction normale et la faire aussi longue que nécessaire.

II) VARIABLES LOCALES, VARIABLES GLOBALES :

Lorsqu'une fonction est appelée, Python réserve pour elle (dans la mémoire de l'ordinateur) **un espace de noms**. Cet espace de noms **local** à la fonction est à distinguer de l'espace de noms **global** où se trouvait les variables du programme principal. Dans l'espace de noms local, nous aurons des variables qui ne sont accessibles qu'au sein de la fonction. C'est par exemple le cas des variables **start**, **stop**, **step** et i dans l'exemple précédent.
A chaque fois que nous définissons des variables à l'intérieur du corps d'une fonction, ces variables ne sont accessibles qu'à la fonction elle-même. On dit que ces variables sont des **variables locales** à la fonction. Une variable locale peut avoir le même nom qu'une variable de l'espace de noms global mais elle reste néanmoins indépendante.
Les contenus des variables locales sont stockés dans l'espace de noms local qui est inaccessible depuis l'extérieur de la fonction.
Les variables définies à l'extérieur d'une fonction sont des variables globales. Leur contenu est « visible » de l'intérieur d'une fonction, mais la fonction ne peut pas le modifier.

Exemple :

```
def test():
    b = 5
    print(a, b)
a = 2
b = 7
test()
print(a, b)
```

Affichage après exécution :

```
>>>
2 5
2 7
```

Utilisation d'une variable globale - global
Il peut se faire par exemple que vous ayez à définir une fonction qui soit capable de modifier une variable globale. Pour atteindre ce résultat, il vous suffira d'utiliser l'instruction **global**. Cette instruction permet d'indiquer - à l'intérieur de la définition d'une fonction - quelles sont les variables à traiter globalement. On va ici créer une fonction qui a accès à la variable globale **b**.

```
def test():
    global b
    b = 5
    print(a, b)
a = 2
b = 7
test()
print(a, b)
```

Affichage après exécution :

```
>>>
2 5
2 5
```

III) FONCTIONS ET PROCEDURES :

Pour les puristes, les fonctions que nous avons décrites jusqu'à présent ne sont pas tout à fait des fonctions au sens strict, mais plus exactement des procédures. Une « vraie » fonction (au sens strict) doit en effet renvoyer une valeur lorsqu'elle se termine. Une « vraie » fonction peut s'utiliser à la droite du signe égale dans des expressions telles que y = sin(a). On comprend aisément que dans cette expression, la fonction sin() renvoie une valeur (le sinus de l'argument) qui est directement affectée à la variable y.

Voici un exemple extrêmement simple :

```
def cube(w):
    return w**3
```

```
>>> cube(3)
27
>>> a = cube(4)
>>> a
64
```

IV) VALEURS PAR DEFAUT POUR LES PARAMETRES :

Dans la définition d'une fonction, il est possible de définir un argument par défaut pour chacun des paramètres.
On obtient ainsi une fonction qui peut être appelée avec une partie seulement des arguments attendus.

Exemples :

```
def politesse(nom, titre ="Monsieur"):
    print("Veuillez agréer,", titre, nom, ", mes salutations distinguées.")
```

```
>>> politesse("SKYWALKER")
Veuillez agréer, Monsieur SKYWALKER , mes salutations distinguées.
>>> politesse('Sara', 'Mademoiselle')
Veuillez agréer, Mademoiselle Sara , mes salutations distinguées.
```

Lorsque l'on appelle cette fonction en ne lui fournissant que le premier argument, le second reçoit tout de même une valeur par défaut. Si l'on fournit les deux arguments, la valeur par défaut pour le deuxième est tout simplement ignorée.

V) ARGUMENTS AVEC ETIQUETTES :

Dans la plupart des langages de programmation, les arguments que l'on fournit lors de l'appel d'une fonction doivent être fournis exactement dans le même ordre que celui des paramètres qui leur correspondent dans la définition de la fonction.
Python autorise cependant une souplesse beaucoup plus grande. Si les paramètres annoncés dans la définition de la fonction ont reçu chacun une valeur par défaut, sous la forme déjà décrite ci-dessus, on peut faire appel à la fonction en fournissant les arguments correspondants **dans n'importe quel ordre, à la condition de désigner nommément les paramètres correspondants**.

Exemple :

```
def oiseau(voltage=100, action="danser la java"):
    print("Ce perroquet ne pourra pas", action)
    print("si vous le branchez sur", voltage, "volts !")
```

```
>>> oiseau(voltage=250, action="vous approuver")
Ce perroquet ne pourra pas vous approuver
si vous le branchez sur 250 volts !
>>> oiseau()
Ce perroquet ne pourra pas danser la java
si vous le branchez sur 100 volts !
```

VI) UTILISATION DES FONCTIONS DANS UN SCRIPT :

Pour cette première approche des fonctions, nous n'avons utilisé jusqu'ici que des fonctions dans des scripts et pas de programme principal dans le script. Veuillez donc essayer vous-même le petit programme ci-dessous, lequel calcule le volume d'une sphère à l'aide de la formule : $V = (4/3)*\pi*R^3$.

Exemple :

```
import numpy as np
def cube(n):
    return n**3
def volume_sphere(r):
```

```
    return 4 / 3 * np.pi * cube(r)
r = float(input("Entrez la valeur du rayon : "))
print("Le volume de cette sphere vaut", volume_sphere(r))
```

A bien y regarder, ce programme comporte deux parties :

- Les deux fonctions cube() et volume_sphere()
- Le corps principal du programme.

Dans le corps principal du programme, il y a un appel de la fonction volume_sphere(). A l'intérieur de la fonction volume_sphere(), il y a un appel de la fonction cube().

Notez bien que les deux parties du programme ont été disposées dans un certain ordre :

- D'abord la définition des fonctions,
- Et ensuite le corps principal du programme.

Cette disposition est nécessaire, parce que l'interpréteur exécute les lignes d'instructions du programme l'une après l'autre, dans l'ordre où elles apparaissent dans le code source. Dans le script, la définition des fonctions doit donc précéder leur utilisation.
En fait, le corps principal d'un programme Python constitue lui-même une entité un peu particulière, qui est toujours reconnue dans le fonctionnement interne de l'interpréteur sous le nom réservé __main__ (le mot « main » signifie « principal », en anglais. Il est encadré par des caractères « souligné » en double, pour éviter toute confusion avec d'autres symboles). L'exécution d'un script commence toujours avec la première instruction de cette entité __main__, où qu'elle puisse se trouver dans le listing. Les instructions qui suivent sont alors exécutées l'une après l'autre, dans l'ordre, jusqu'au premier appel de fonction. Un appel de fonction est comme un détour dans le flux de l'exécution : au lieu de passer à l'instruction suivante, l'interpréteur exécute la fonction appelée, puis revient au programme appelant pour continuer le travail interrompu. Pour que ce mécanisme puisse fonctionner, il faut que l'interpréteur ait pu lire la définition de la fonction avant l'entité __main__, et celle-ci sera donc placée en général à la fin du script.
Dans notre exemple, l'entité __main__ appelle une première fonction qui elle-même en appelle une deuxième.
Cette situation est très fréquente en programmation. Si vous voulez comprendre correctement ce qui se passe dans un programme, vous devez donc apprendre à lire un script, non pas de la première à la dernière ligne, mais plutôt en suivant un cheminement analogue à ce qui se passe lors de l'exécution de ce script. Cela signifie concrètement que vous devrez souvent analyser un script en commençant par ses dernières lignes !

En référence à l'exemple précédent, on pourra donc écrire :

```
import numpy as np

def cube(n):
    return n**3
def volume_sphere(r):
    return 4 / 3 * np.pi * cube(r)

def main(): # Corps du programme principal
    r = float(input("Entrez la valeur du rayon : "))
    print("Le volume de cette sphere vaut", volume_sphere(r))
    pass

if __name__ == '__main__':
    main()
```

<u>Rem</u> : contrairement à d'autres langages, où ce type de présentation utilisant main est obligatoire, Python ne l'exige pas et peut très bien, la plupart du temps, s'en passer.

Résumé : structure d'un programme Python type

```
# -*- coding:Utf8 -*-

####################################
# Programme Python type            #
# auteur : G.Swinnen, Liège, 2009  #
# licence : GPL                    #
####################################


#####################################
# Importation de fonctions externes :

from math import sqrt


##################################
# Définition locale de fonctions :

def occurrences(car, ch):
    "Cette fonction renvoie le \
     nombre de caractères <car> \
     contenus dans la chaîne <ch>"

    nc = 0

    i = 0

    while i < len(ch):

        if ch[i] == car:
            nc = nc + 1

        i = i + 1

    return nc


################################
# Corps principal du programme :

print("Veuillez entrer un nombre :")
nbr = eval(input())

print("Veuillez entrer une phrase :")
phr = input()
print("Entrez le caractère à compter :")
cch = input()

no = occurrences(cch, phr)
rc = sqrt(nbr**3)

print("La racine carrée du cube", end=' ')
print("du nombre fourni vaut", end=' ')
print(rc)

print("La phrase contient", end=' ')
print(no, "caractères", cch)
```

Un programme Python contient en général les blocs suivants, dans l'ordre :
- Quelques instructions d'initialisation (importation de fonctions et/ou de classes, définition éventuelle de variables globales).
- Les définitions locales de fonctions et/ou de classes.
- Le corps principal du programme.

Le programme peut utiliser un nombre quelconque de fonctions, lesquelles sont définies localement ou importées depuis des modules externes.
Vous pouvez vous-même définir de tels modules.

La définition d'une fonction comporte souvent une liste de PARAMÈTRES.
Ce sont toujours des VARIABLES, qui recevront leur valeur lorsque la fonction sera appelée.

Une boucle de répétition de type 'while' doit toujours inclure au moins quatre éléments :
- l'initialisation d'une variable 'compteur' ;
- l'instruction while proprement dite, dans laquelle on exprime la condition de répétition des instructions qui suivent ;
- le bloc d'instructions à répéter ;
- une instruction d'incrémentation du compteur.

La fonction "renvoie" toujours une valeur bien déterminée au programme appelant.
Si l'instruction 'return' n'est pas utilisée, ou si elle est utilisée sans argument, la fonction renvoie un objet vide : 'None'.

Le programme qui fait appel à une fonction lui transmet d'habitude une série d'ARGUMENTS, lesquels peuvent être des valeurs, des variables, ou même des expressions.

VI) <u>APPLICATIONS</u> :

Pour certains de ces exercices, la notion de liste est utile. En attendant l'activité sur les listes, se reporter au cours sur le site Pre NSI DDM45, et plus particulièrement aux instructions len() et append. Pour plus d'informations :
https://python-django.dev/page-apprendre-listes-list-tableaux-tableaux-liste-array-python-cours-debutant

<u>Exercice 1</u> :
Définissez une fonction **surfCercle(R)**. Cette fonction doit renvoyer la surface (l'aire) d'un cercle dont on lui a fourni le rayon **R** en argument. Par exemple, l'exécution de l'instruction :
print(surfCercle(2.5)) doit donner le résultat : **19.63495...**

<u>Exercice 2</u> :
Définissez une fonction **volBoite(x1,x2,x3)** qui renvoie le volume d'une boîte parallélépipédique dont on fournit les trois dimensions **x1, x2, x3** en arguments.
Par exemple, l'exécution de l'instruction :
print(volBoite(5.2, 7.7, 3.3)) doit donner le résultat : **132.132.**

<u>Exercice 3</u> :
Définissez une fonction **maximum(n1,n2,n3)** qui renvoie le plus grand de 3 nombres **n1, n2, n3** fournis en arguments. Par exemple, l'exécution de l'instruction :
print(maximum(2,5,4)) doit donner le résultat : **5.**

Exercice 4 :
Définissez une fonction **compteCar(ca,ch)** qui renvoie le nombre de fois que l'on rencontre le caractère **ca** dans la chaîne de caractères **ch**. Par exemple, l'exécution de l'instruction :
print(compteCar('e', 'Cette phrase est un exemple')) doit donner le résultat : **7**

Exercice 5 :
Définissez une fonction **indexMax(liste)** qui renvoie l'index de l'élément ayant la valeur la plus élevée dans la liste transmise en argument. Exemple d'utilisation :
serie = [5, 8, 2, 1, 9, 3, 6, 7]
print(indexMax(serie))
4

Exercice 6 :
Définissez une fonction **nomMois(n)** qui renvoie le nom du énième mois de l'année.
Par exemple, l'exécution de l'instruction :
print(nomMois(4)) doit donner le résultat : **Avril.**

Exercice 7 :
Modifiez la fonction **volBoite(x1,x2,x3)** que vous avez définie dans un exercice précédent, de manière à ce qu'elle puisse être appelée avec trois, deux, un seul, ou même aucun argument. Utilisez pour ceux-ci des valeurs par défaut égales à 10.
Par exemple :
print(volBoite()) doit donner le résultat : **1000**
print(volBoite(5.2)) doit donner le résultat : **520.0**
print(volBoite(5.2, 3)) doit donner le résultat : **156.0**

Exercice 8 :
Modifiez la fonction **volBoite(x1,x2,x3)** ci-dessus de manière à ce qu'elle puisse être appelée avec un, deux, ou trois arguments. Si un seul est utilisé, la boîte est considérée comme cubique (l'argument étant l'arête de ce cube). Si deux sont utilisés, la boîte est considérée comme un prisme à base carrée (auquel cas le premier argument est le côté du carré, et le second la hauteur du prisme). Si trois arguments sont utilisés, la boîte est considérée comme un parallélépipède.
Par exemple :
print(volBoite()) doit donner le résultat : **-1** (indication d'une erreur)
print(volBoite(5.2)) doit donner le résultat : **140.608**
print(volBoite(5.2, 3)) doit donner le résultat : **81.12**
print(volBoite(5.2, 3, 7.4)) doit donner le résultat : **115.44**

Exercice 9 :
Définissez une fonction **changeCar(ch,ca1,ca2,debut,fin)** qui remplace tous les caractères **ca1** par des caractères **ca2** dans la chaîne de caractères **ch**, à partir de l'indice **debut** et jusqu'à l'indice **fin**, ces deux derniers arguments pouvant être omis (et dans ce cas la chaîne est traitée d'une extrémité à l'autre). Exemples de la fonctionnalité attendue :

```
>>> phrase = 'Ceci est une toute petite phrase.'
>>> print(changeCar(phrase, ' ', '*'))
Ceci*est*une*toute*petite*phrase.
>>> print(changeCar(phrase, ' ', '*', 8, 12))
Ceci est*une*toute petite phrase.
>>> print(changeCar(phrase, ' ', '*', 12))
Ceci est une*toute*petite*phrase.
>>> print(changeCar(phrase, ' ', '*', fin = 12))
Ceci*est*une*toute petite phrase.
```

Exercice 10 :
Définissez une fonction **eleMax(liste,debut,fin)** qui renvoie l'élément ayant la plus grande valeur dans la liste transmise. Les deux arguments **debut** et **fin** indiqueront les indices entre lesquels doit s'exercer la recherche, et chacun d'eux pourra être omis (comme dans l'exercice précédent). Exemples de la fonctionnalité attendue :

```
>>> serie = [9, 3, 6, 1, 7, 5, 4, 8, 2]
>>> print(eleMax(serie))
9
>>> print(eleMax(serie, 2, 5))
7
```

```
>>> print(eleMax(serie, 2))
8
>>> print(eleMax(serie, fin =3, debut =1))
```

Exercice 11 :
Écrire un programme qui permet de jouer au jeu du « plus petit, plus grand» : l'ordinateur choisi un nombre entre 0 et 100. Le but du joueur est de deviner ce nombre. Pour cela, il propose un nombre, et le programme répond si le nombre proposé est plus petit ou plus grand que le nombre choisi. On recommence ainsi jusqu'à ce que le joueur ait trouvé le nombre.
Bonus : on pourra compter le nombre de coups qu'il a fallu au joueur pour trouver le nombre.
Remarque : pour choisir un nombre entier x au hasard entre 0 et 100, on utilisera :

```
from random import randint
x = randint(0, 100)
```

Exercice 12 :
Chercher ce qu'est la suite de Syracuse d'un nombre. Écrire des fonctions qui :
• renvoie le terme suivant d'un terme donné ;
• affiche la suite de Syracuse d'un nombre ;
• renvoie le temps de vol d'un nombre ;
• renvoie l'altitude maximale d'un nombre.

Exercice 13 : (qui pourra être mieux compris lorsque le cours sur la numération sera traité, pensez-y le moment venu)
Ecrire un programme qui prend en entrée deux nombres écrits en binaire (sur 8 bits) et qui retourne la somme (en binaire) de ces deux nombres. (Additionneur binaire).